AUTHENTICATION

Authentifizierung, bcrypt, Cookies, Session, OAuth

Stephanie Held
Studiengang Medieninformatik
Hochschule für Angewandte Wissenschaften Hamburg

HAW HAMBURG

LERNZIELE

Nach dieser Vorlesung könnt ihr

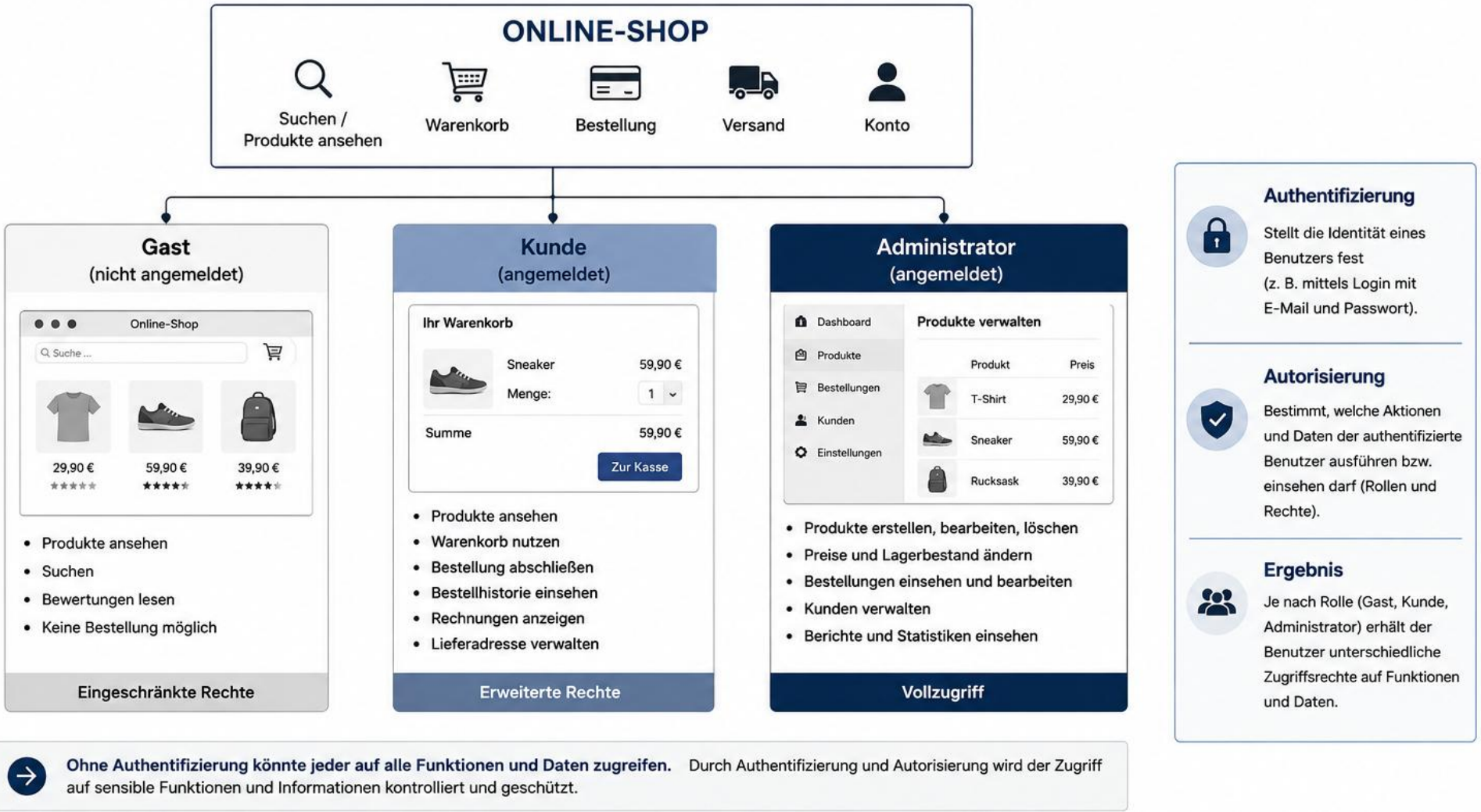
- Authentifizierungsverfahren unterscheiden
- Passwörter sicher speichern
- Session-Management erläutern
- Passport.js konfigurieren
- OAuth 2.0 erklären
- Rollen- und Rechtekonzepte anwenden

AUTHENTIFIZIERUNG UND AUTORISIERUNG IN WEBANWENDUNGEN

"Security is not a feature – it is a requirement."

HAW HAMBURG

WARUM BENÖTIGEN WIR AUTHENTIFIZIERUNG?



WARUM BENÖTIGEN WEBANWENDUNGEN AUTHENTIFIZIERUNG?

Nahezu alle modernen Webanwendungen verarbeiten personenbezogene oder vertrauliche Daten. Ohne eine zuverlässige Identitätsprüfung könnten Benutzer auf Daten anderer Personen zugreifen oder Aktionen unter fremder Identität ausführen.

Typische Anwendungsbeispiele sind:

- Online-Banking
- Lernplattformen (z. B. Moodle)
- Cloud-Dienste
- E-Commerce
- Soziale Netzwerke

DEFINITION: AUTHENTIFIZIERUNG

Definition

Die **Authentifizierung** beschreibt den Prozess der Überprüfung einer behaupteten Identität. Ziel ist es festzustellen, ob ein Benutzer, ein Dienst oder ein System tatsächlich die Identität besitzt, die gegenüber einer Anwendung angegeben wurde.
In Webanwendungen erfolgt die Authentifizierung häufig über Benutzername und Passwort. Moderne Systeme unterstützen zusätzlich Verfahren wie Mehrfaktorauthentifizierung (MFA), biometrische Merkmale oder föderierte Identitäten (z.B. Mit Google/Microsoft/Apple anmelden).

Beispiele

- Benutzername + Passwort
- Einmalpasswort (OTP)
- Fingerabdruck
- Face ID
- Sicherheitsschlüssel (FIDO2/WebAuthn)

DEFINITION: AUTORISIERUNG

Definition

Die **Autorisierung** beschreibt den Prozess der Überprüfung, auf welche Ressourcen oder Funktionen ein bereits authentifizierter Benutzer zugreifen darf. Ziel ist es sicherzustellen, dass Benutzer ausschließlich die Aktionen ausführen können, für die ihnen entsprechende Berechtigungen erteilt wurden.
Die Autorisierung erfolgt **nach** der erfolgreichen Authentifizierung und basiert häufig auf Rollen (*Role-Based Access Control, RBAC*), Berechtigungen (*Permissions*) oder Richtlinien (*Policies*).

Beispiele

- Administrator darf Benutzer verwalten.
- Dozierende dürfen Noten eintragen.
- Studierende dürfen nur ihre eigenen Prüfungsleistungen einsehen.
- Ein Gast darf öffentliche Inhalte lesen, aber keine Änderungen vornehmen.
- Ein Online-Shop erlaubt nur angemeldeten Benutzern den Abschluss einer Bestellung.

WARUM SIND AUTHENTIFIZIERUNG UND AUTORISIERUNG NOTWENDIG?

Authentifizierung und **Autorisierung** bilden grundlegende Sicherheitsmechanismen in modernen IT-Systemen. Sie gewährleisten, dass ausschließlich berechtigte Personen auf Informationen und Systeme zugreifen können.

- **Identitätsprüfung:** Sicherstellung der Identität eines Benutzers vor dem Zugriff auf ein System (Authentifizierung).
- **Datenschutz:** Schutz personenbezogener und sensibler Daten vor unbefugtem Zugriff gemäß Datenschutz- und Sicherheitsanforderungen.
- **Nachvollziehbarkeit:** Eindeutige Zuordnung von Aktionen zu authentifizierten Benutzern zur Unterstützung von Audits und der Revisionssicherheit.
- **Schutz kritischer Ressourcen:** Autorisierung stellt sicher, dass Benutzer ausschließlich auf die Ressourcen und Funktionen zugreifen können, für die ihnen entsprechende Berechtigungen zugewiesen wurden.

Kernaussage:
Authentifizierung beantwortet die Frage „Wer bist du?“, während Autorisierung festlegt „Was darfst du tun?“. Gemeinsam bilden sie die Grundlage für Vertraulichkeit, Integrität und sichere Zugriffskontrolle in IT-Systemen.

AUTHENTIFIZIERUNG VS. AUTORISIERUNG



HTTP IST ZUSTANDSLOS

HTTP ist ein **zustandsloses (stateless)** Protokoll. Jede HTTP-Anfrage wird unabhängig von vorherigen Anfragen verarbeitet. Der Server speichert standardmäßig keine Information darüber, welcher Benutzer eine frühere Anfrage gestellt hat.

Konsequenz
Nach einer erfolgreichen Anmeldung weiß der Server bei der nächsten Anfrage zunächst nicht mehr, welcher Benutzer kommuniziert.

Aus diesem Grund werden zusätzliche Mechanismen benötigt:

- Cookies
- Sessions
- JSON Web Tokens (JWT)

Merke: Ohne Zustandsverwaltung müsste sich ein Benutzer bei jeder einzelnen Anfrage erneut anmelden.

AUSBLICK

Wie entsteht ein sicheres Login?
In den folgenden Kapiteln entwickeln wir schrittweise eine sichere Authentifizierung.

Jede Stufe löst ein konkretes Sicherheitsproblem der vorherigen Lösung und erweitert die Anwendung um einen wichtigen Baustein moderner Webanwendungen.



EINFACHES AUTHENTIFIZIEREN

HAW HAMBURG

WIR ENTWICKELN EINEN LOGIN

Stellen wir uns vor, wir entwickeln eine kleine Webanwendung für eine Hochschule.

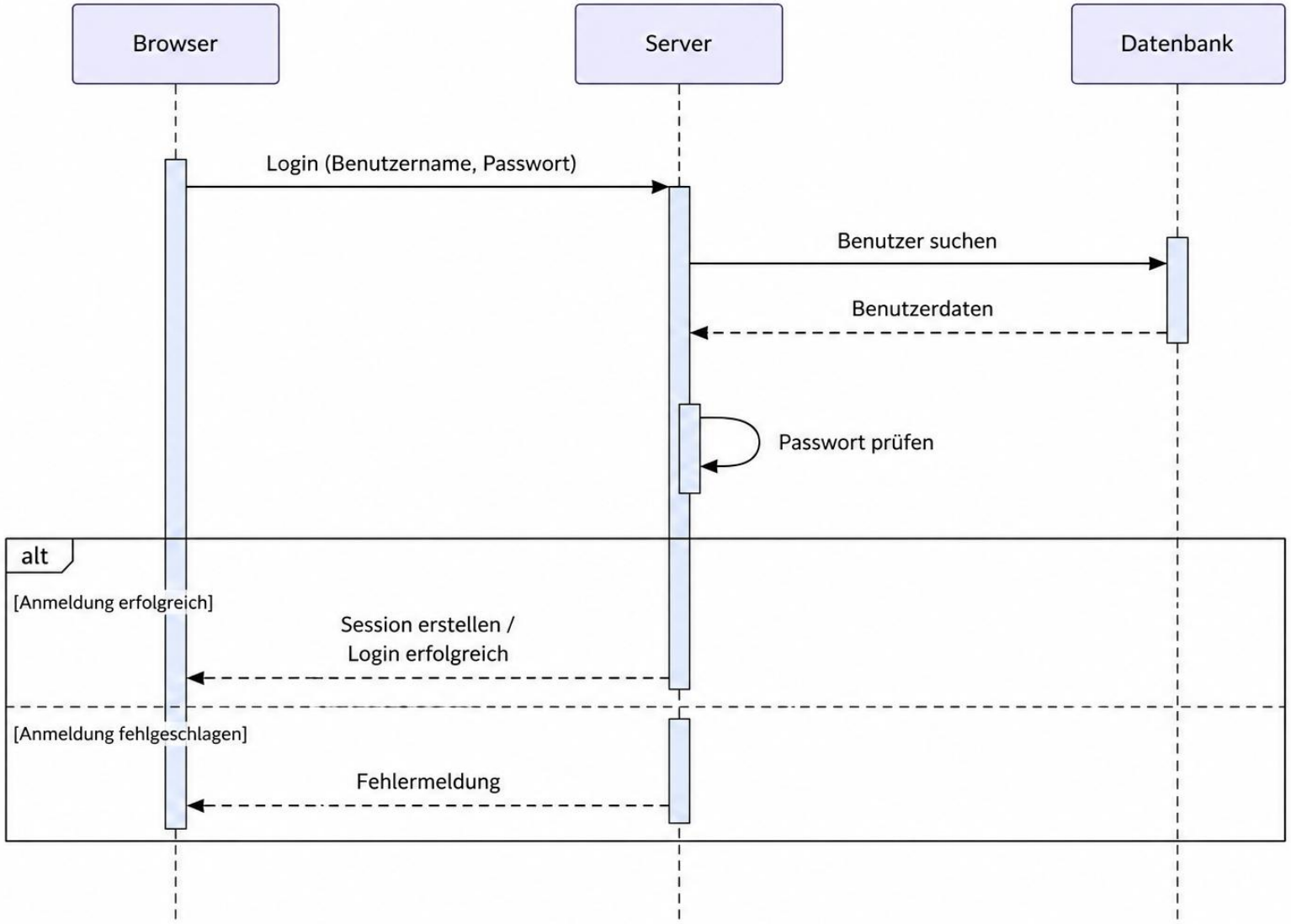
Studierende sollen sich anmelden können, um

- Vorlesungsunterlagen herunterzuladen
- Noten einzusehen
- Übungen abzugeben

Die erste Idee erscheint einfach:
Der Benutzer gibt einen Benutzernamen und ein Passwort ein. Stimmen beide mit den gespeicherten Daten überein, wird der Login akzeptiert.

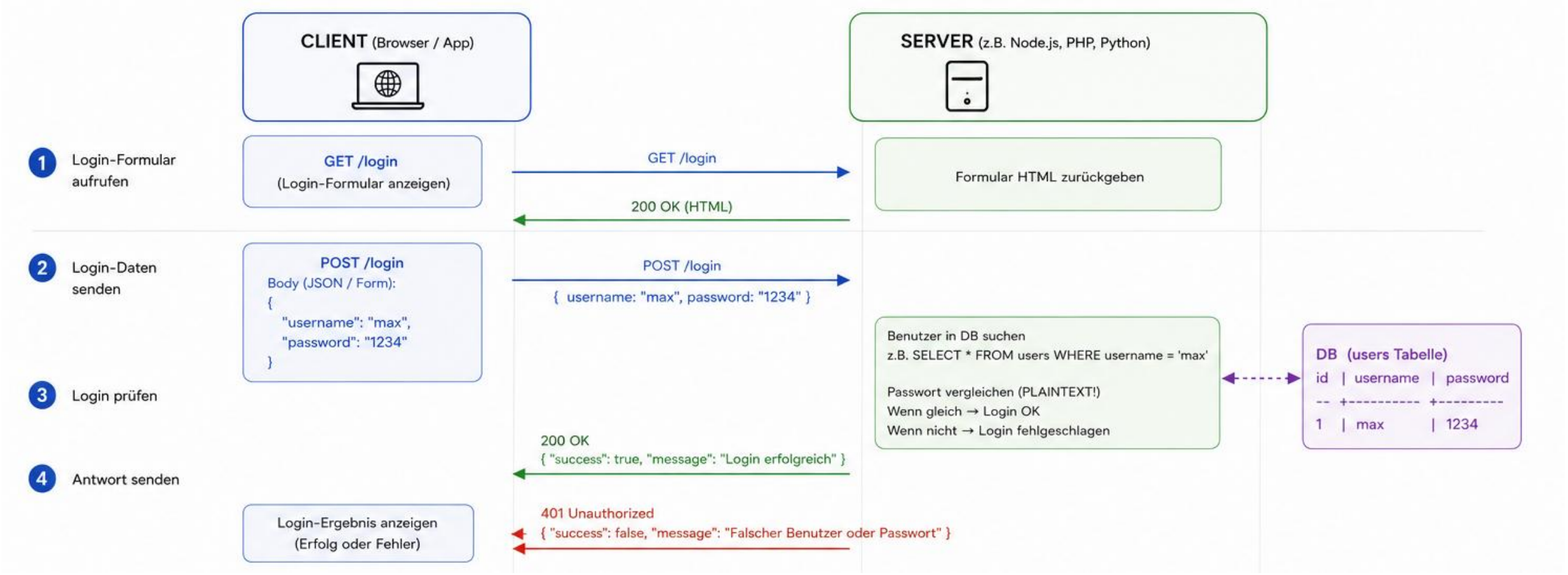
Diskussionsfrage
Reicht das bereits für eine sichere Webanwendung?

AUTHENTIFIZIERUNGSPROZESS



HAW HAMBURG

AUTHENTIFIZIERUNGSPROZESS



LOGIN FRONTEND - HTTP REQUEST

Das Frontend schickt Username und Password als HTTP POST Request an den Server:

```
export async function loginUser(username, password) {
  const response = await fetch(`${API_URL}/login`, {
    method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify({ username, password })
  });

  return response.json();
}
```

LOGIN BACKEND - HTTP RESPONSE

Der Server empfängt den HTTP Request des Clients mit Username und Password:

```
app.post("/api/login", async (req, res) => {
  const { username, password } = req.body;

  if (!username || !password) {
    return res.status(400).json({
      success: false,
      message: "Benutzername und Passwort sind erforderlich"
    });
  }

  const [status, success, message] = await checkLogin(username, password);
  return res.json({
    status: status,
    success: success,
    message: message
  });

});
```

LOGIN BACKEND - DATENBANKZUGRIFF

Der Server macht eine Datenbankabfrage und überprüft ob ein Benutzer mit dem Username und Password existiert:

```
export async function checkLogin(username, password) {
  try {
    const result = await db.query(
      "SELECT id, username FROM users WHERE username = $1 AND password = $2",
      [username, password]
    );

    if (result.rows.length === 0) {
      return [401, false, "Login fehlgeschlagen"];
    }
    return [200, true, "Login erfolgreich"];

  } catch {
    return [500, false, "Fehler beim Login"];
  }

}
```

ÜBUNG: EINFACHE REGISTRIERUNG UND LOGIN

Nehmt die Finkenau-Festival Webseite und erweitert sie um einen User Registrierung und Login:

- Legt eine users-Datenbank Tabelle mit id, email (UNIQUE) und password an.

- Nehmt über die Route „/registration“ email und password an im body an. Gebt zunächst die Daten im body auf der Konsole aus.

- Speichert den Benutzer mit email und password in der Datenbanktabelle users, falls er noch nicht existiert.
  Die Email soll nur mit Kleinbuchstaben gespeichert werden: text.toLowerCase()
  SQL: `INSERT INTO users (email, password) VALUES ($1, $2) ON CONFLICT (email) DO NOTHING;`

ÜBUNG: EINFACHE REGISTRIERUNG UND LOGIN

Nehmt die Finkenau-Festival Webseite und erweitert sie um einen User Registrierung und Login:

- Nehmt über die Route „/login“ email und password an im body an. Gebt zunächst die Daten im body wieder auf der Konsole aus.

- Lasst euch nun alle Benutzer mit der angegebenen Email ausgeben. Konvertiert die Email wieder in Kleinbuchstaben und checkt das Passwort anschließend. Mit `rowCount` könnt ihr die Anzahl der zurückgegebenen Einträge abfragen, mit `rows[0]` könnt ihr ggfs auf das 1. Element des zurückgegebenen Arrays zugreifen.
  SQL: `SELECT * FROM users WHERE email = $1`

- Zeigt auf der Webseite entweder „Login successful“ oder „Email or password incorrect“ an.

SICHERE PASSWORTSPEICHERUNG

HAW HAMBURG

SICHERE PASSWORTSPEICHERUNG

Lernziele

Nach diesem Kapitel könnt ihr

- begründen, warum Passwörter niemals im Klartext gespeichert werden dürfen,
- den Unterschied zwischen Verschlüsselung und Hashing erläutern,
- erklären, warum Passwort-Hashverfahren wie bcrypt eingesetzt werden,
- die Funktion eines Salts beschreiben,
- Passwörter mit bcrypt hashen und überprüfen.

AUSGANGSSITUATION

Unser Login funktioniert – aber ist er sicher?

Im vorherigen Kapitel haben wir einen ersten Login implementiert.

Der Server sucht den Benutzer und vergleicht anschließend das gespeicherte Passwort mit der Eingabe.

```
const result = await db.query(
  "SELECT id, username FROM users WHERE username = $1 AND password = $2",
  [username, password]
);
```

Aus funktionaler Sicht erfüllt diese Lösung ihren Zweck. Aus sicherheitstechnischer Sicht weist sie jedoch erhebliche Schwächen auf.

Problem

Jeder Benutzer besitzt ein im Klartext gespeichertes Passwort. Erhält ein Angreifer Zugriff auf die Datenbank, sind sämtliche Zugangsdaten unmittelbar lesbar.

	id [PK] integer	username text	password text
1	3	hallo@gmail.co...	1234

AUSGANGSSITUATION

Echtes Szeario

WARUM IST DAS PROBLEMATISCH?

Was passiert bei einem Datenbank-Leak?
Angenommen, ein Angreifer erhält Zugriff auf die Benutzerdatenbank.

	id [PK] integer	username text	password text
1	3	hallo@gmail.co...	1234

Der Angreifer benötigt keinerlei zusätzliche Werkzeuge. Die Passwörter sind sofort bekannt.

Die eigentliche Gefahr besteht jedoch darin, dass viele Benutzer Passwörter mehrfach verwenden. Ein kompromittiertes Passwort wird deshalb häufig automatisiert auch bei anderen Diensten getestet. Dieser Angriff wird als **Credential Stuffing** bezeichnet.

Merke: Klartextpasswörter gefährden nicht nur die eigene Anwendung, sondern häufig auch Benutzerkonten bei anderen Diensten.

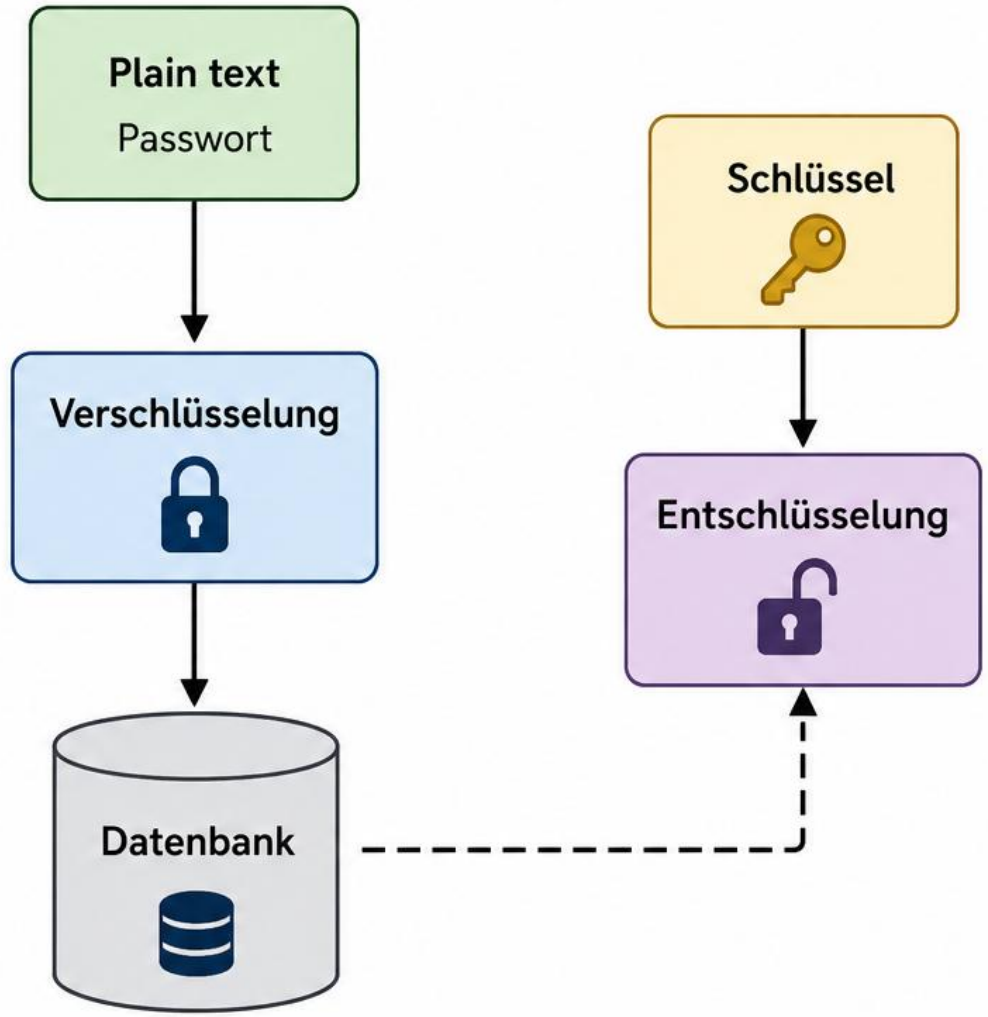
KÖNNTEN WIR PASSWÖRTER VERSCHLÜSSELN?

Diese Idee liegt zunächst nahe:

"Anstatt das Passwort im Klartext zu speichern, verschlüsseln wir es einfach.„

Auf den ersten Blick scheint dies eine sinnvolle Lösung zu sein.

Zum Vergleich beim Login würde der Server das Passwort wieder entschlüsseln und mit der Benutzereingabe vergleichen.
Doch genau hier liegt das Problem.

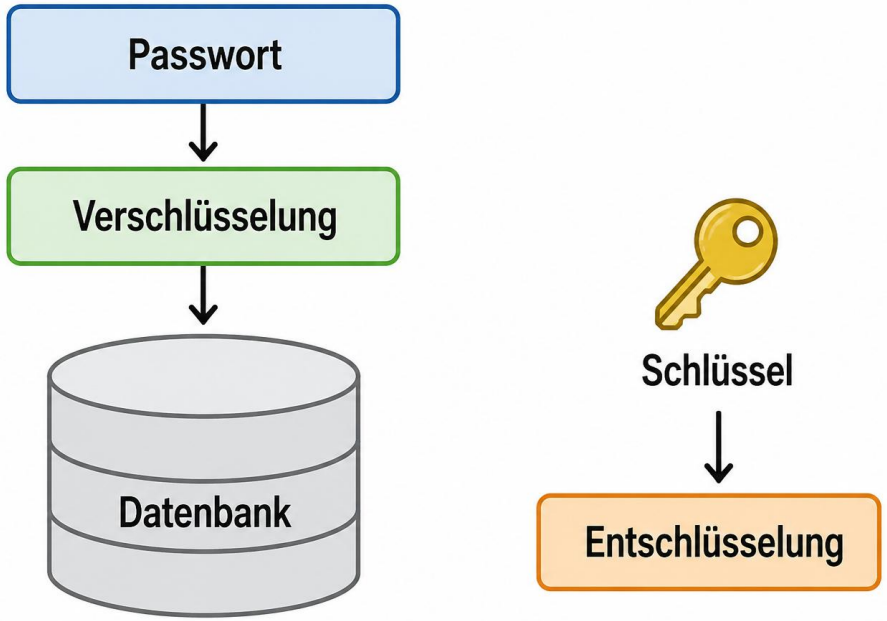


WARUM VERSCHLÜSSELUNG KEINE GUTE LÖSUNG IST

Eine Verschlüsselung ist grundsätzlich **umkehrbar**. Damit ein verschlüsseltes Passwort wieder gelesen werden kann, benötigt die Anwendung einen geheimen Schlüssel.

Befindet sich dieser Schlüssel auf dem Server – was für den Login notwendig wäre –, kann ein Angreifer bei einem erfolgreichen Angriff unter Umständen sowohl auf die Datenbank als auch auf den Schlüssel zugreifen.
In diesem Fall könnten sämtliche Passwörter entschlüsselt werden.

Merke: Passwörter müssen nicht gelesen, sondern lediglich überprüft werden. Deshalb ist Verschlüsselung für die Passwortspeicherung ungeeignet.



HASHING ALS LÖSUNG

Bei der Passwortspeicherung verfolgen wir einen anderen Ansatz.

Anstatt das Passwort zu verschlüsseln, berechnet die Anwendung einen Hashwert.

Der entscheidende Unterschied besteht darin, dass der Hash **nicht zurückgerechnet** wird.
Beim Login wird stattdessen das neu eingegebene Passwort erneut gehasht und mit dem gespeicherten Hash verglichen.
Dadurch muss die Anwendung das ursprüngliche Passwort niemals speichern oder wiederherstellen.

WARUM VERWENDEN WIR NICHT SHA-256?

Die meisten von euch kennen bestimmt SHA-256 aus anderen Veranstaltungen, aus der Praxis oder dem Alltag. Daher liegt die Vermutung nahe, SHA-256 auch für Passwörter einzusetzen.

Für diesen Einsatzzweck wurde SHA-256 jedoch nicht entwickelt.

SHA-256 ist eine allgemeine kryptographische Hashfunktion, deren Ziel eine möglichst schnelle Berechnung ist. Moderne Hardware kann dadurch Millionen bis Milliarden Hashwerte pro Sekunde berechnen. Genau das erleichtert Brute-Force- und Wörterbuchangriffe auf gestohlene Passwortdatenbanken.

Passwort-Hashverfahren wie **bcrypt** oder **Argon2id** verfolgen einen anderen Ansatz: Sie erhöhen den Rechenaufwand jeder Passwortprüfung bewusst. Dadurch wird jeder einzelne Passwortversuch deutlich teurer.

Merke: Schnelle Hashfunktionen eignen sich gut zur Integritätsprüfung von Daten. Für Passwörter werden bewusst langsame Verfahren verwendet.

PASSWORT-HASHING MIT BCRYPT

In Node.js wird häufig die Bibliothek bcrypt verwendet.
npm install bcrypt

Ein Passwort kann anschließend gehasht werden.

Der zweite Parameter (12) ist der sogenannte **Cost Factor**. Er bestimmt, wie aufwendig die Berechnung des Hashwertes durchgeführt wird. Ein höherer Wert erhöht die Sicherheit, benötigt jedoch auch mehr Rechenzeit.

WARUM UNTERSCHEIDEN SICH DIE HASHWERTE?

```
import bcrypt from "bcrypt";

const a = await bcrypt.hash("123456",12);
const b = await bcrypt.hash("123456",12);
```

Ausgabe:

```
Die Hashwerte für 123456 sind:

$2b$12$Jp/v/Vgy2SpNzPvtlwr//u9UU1vDftrPC8Ws.JZazrj4pvWOD6Lpe
$2b$12$8m0TlR0jlNf99XrA2dTN3u6CSIt1t5zFenVy8640cMGSeuX8xMh8G
```

Obwohl beide Passwörter identisch sind, entstehen unterschiedliche Hashwerte.

Der Grund ist ein **Salt**, das bcrypt automatisch erzeugt und in den Hash integriert.
Das Salt verhindert, dass identische Passwörter zu identischen Hashwerten führen und erschwert vorberechnete Angriffe erheblich.

WAS IST EIN SALT?

Ein Salt ist eine zufällig erzeugte Zeichenfolge, die vor dem Hashing mit dem Passwort kombiniert wird.
Ohne Salt würden zwei Benutzer mit demselben Passwort auch denselben Hash besitzen.

Benutzer	Passwort	Hash
Anna	123456	abc123
Max	123456	abc123

Mit Salt entsteht für jeden Benutzer ein anderer Hash.

Benutzer	Passwort	Hash
Anna	X7kP2	4ab9...
Max	M3qLs	c28f...

Bei bcrypt muss das Salt nicht manuell verwaltet werden. Es wird automatisch erzeugt und zusammen mit dem Hash gespeichert.

PASSWORTPRÜFUNG

Beim Login liegt das Passwort des Benutzers erneut im Klartext vor.
Die Anwendung berechnet daraus keinen eigenen Vergleichswert, sondern verwendet bcrypt.compare().

```
const ok = await bcrypt.compare(
    password,
    hashedPassword
);
```

bcrypt.compare() liest den gespeicherten Salt aus dem Hash, berechnet daraus einen neuen Hash und vergleicht beide Werte.

Dadurch kann das Passwort überprüft werden, ohne jemals dauerhaft gespeichert zu werden.

ZUSAMMENFASSUNG

Das haben wir gelernt

- Passwörter dürfen niemals im Klartext gespeichert werden.
- Verschlüsselung ist für die Passwortspeicherung ungeeignet, da sie eine Entschlüsselung erfordert.
- Stattdessen werden Passwort-Hashverfahren eingesetzt.
- Moderne Anwendungen verwenden dafür Bibliotheken wie bcrypt oder Argon2id.
- Ein Salt sorgt dafür, dass identische Passwörter unterschiedliche Hashwerte erzeugen.
- Mit bcrypt.compare() können Passwörter überprüft werden, ohne dass das ursprüngliche Passwort gespeichert werden muss.

BCRYPT

Nun lasst uns das Passwort verschlüsselt in der Datenbank speichern:

- Installiert mit npm bcrypt
- Importiert `bcrypt from "bcrypt"`
- Erstellt eine Konstante für die saltRounds
- Verändert `register` so, dass ihr den gehashten Wert in die DB ablegt:

```
bcrypt.hash(password, saltRounds, async (err, hash) =>{
//auf err checken und ansonsten user in DB speichern
});
```

BCRYPT

Nun lasst uns das Passwort verschlüsselt in der Datenbank speichern:

- Beim Login soll „`Email or passoword incorrect`“ zurück gegeben werden, wenn die Email nicht gefunden wurde.

- Wenn der User mit der Email vorhanden ist, wird das eingegebene Passwort mit dem Passwort in der DB verglichen:

```
bcrypt.compare(loginPassword, storedHashedPassword, (err, result) => {
//err checke
//wenn die Passwörter übereinstimmen (result = true), dann soll die Nachricht „Login successful“
ausgegeben werden
//ansonsten „Email or passoword incorrect“

})
```

Achtung: Compare gibt ein Promise zurück

REDIRECT AN /ADDPROJECT

Wenn der User eingeloggt ist, soll er nun mit redirect(„/addProject“) zu einer Webseite weitergeleitet werden, bei der der User das Projekt eingeben soll.

SITZUNGSVERWALTUNG

HAW HAMBURG

SITZUNGSVERWALTUNG MIT SESSIONS UND COOKIES

Lernziele

Nach diesem Kapitel könnt ihr:

- erklären, warum ein Login allein nicht ausreicht,
- den Zusammenhang zwischen HTTP und Sessions erläutern,
- Sessions und Cookies unterscheiden,
- den Ablauf einer Session beschreiben,
- Sessions mit Express implementieren,
- Cookies sicher konfigurieren.

MOTIVATION

Warum verliert unser Benutzer den Login?

Szenario

Ihr möchtet mit euren Freunden zum **Wacken Open Air**.
Nach langer Wartezeit liegen endlich vier Tickets im Warenkorb.
Kurz vor dem Bezahlen drückt ihr versehentlich **F5**.

Nach dem Neuladen:

- Benutzer ausgeloggt
- Warenkorb leer
- Tickets ausverkauft

Frage

Warum "vergisst" die Anwendung den Benutzer?



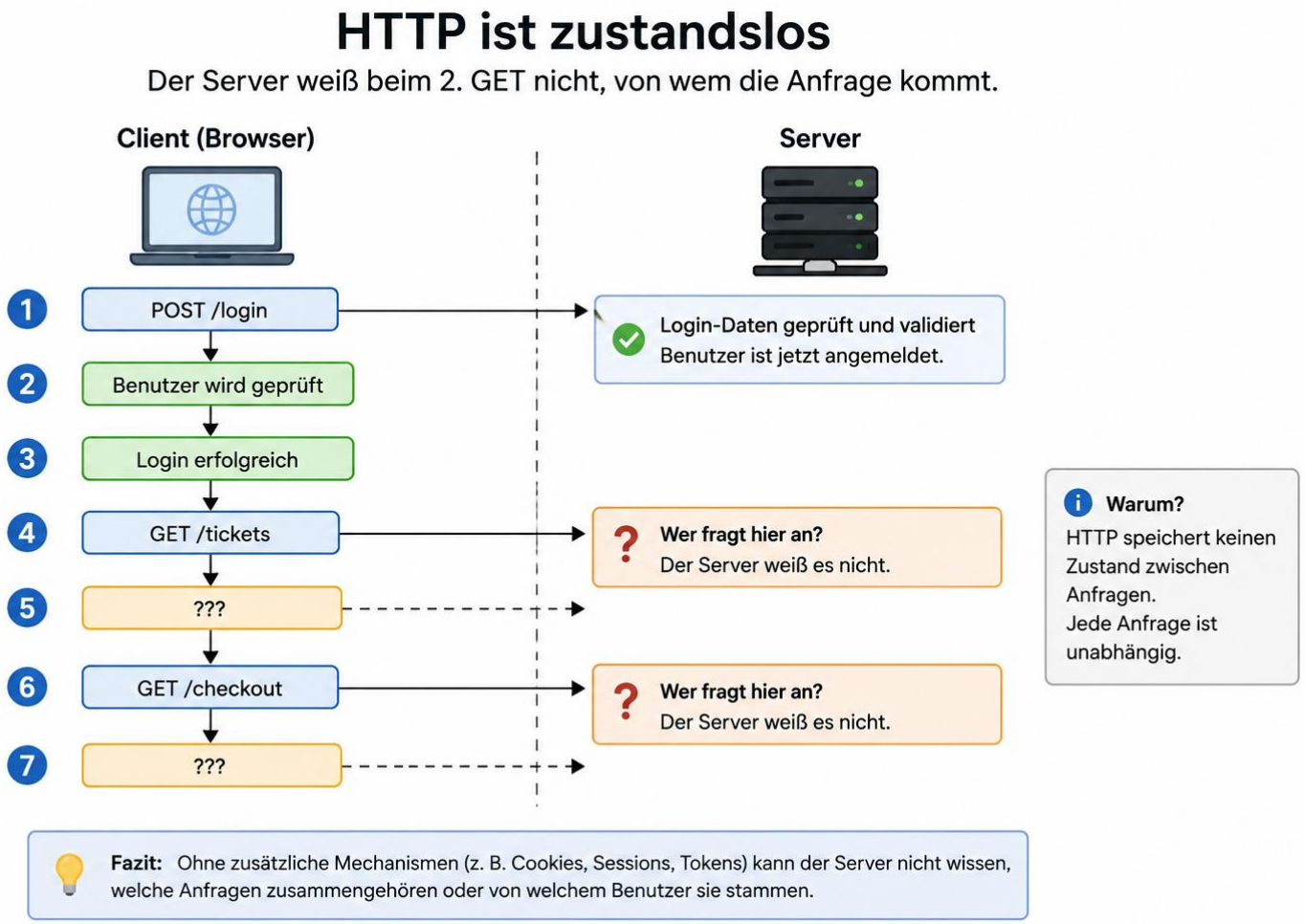
ANALYSE

Der Login funktioniert doch …

Die erste Anfrage funktioniert problemlos.

Aber jede weitere Anfrage besitzt dasselbe Problem:

Woher weiß der Server, welcher Benutzer diese Anfrage sendet?

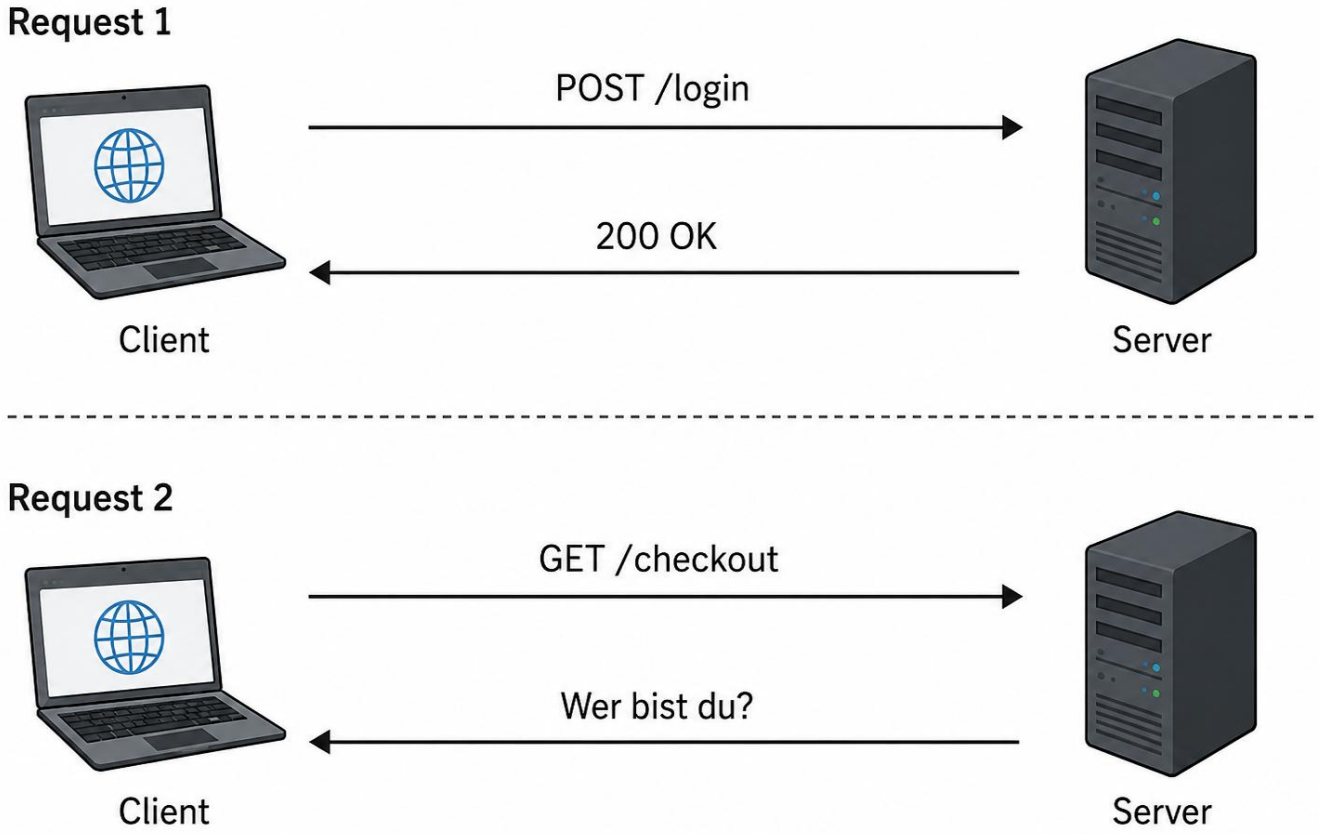


HTTP IST ZUSTANDSLOS

Stateless Protocol
HTTP verarbeitet jede Anfrage unabhängig.

Zwischen zwei Requests existiert **keine eingebaute Verbindung**.

Der Server erinnert sich standardmäßig nicht an vorherige Anfragen.



ERSTE IDEE

Der Server merkt sich den Login

Eine naheliegende Lösung wäre:
Der Server speichert einfach alle angemeldeten Benutzer.

Das funktioniert zunächst.

Aber …
Beim nächsten Request erhält der Server lediglich
GET /checkout
Welche der drei Anmeldungen gehört zu dieser Anfrage?

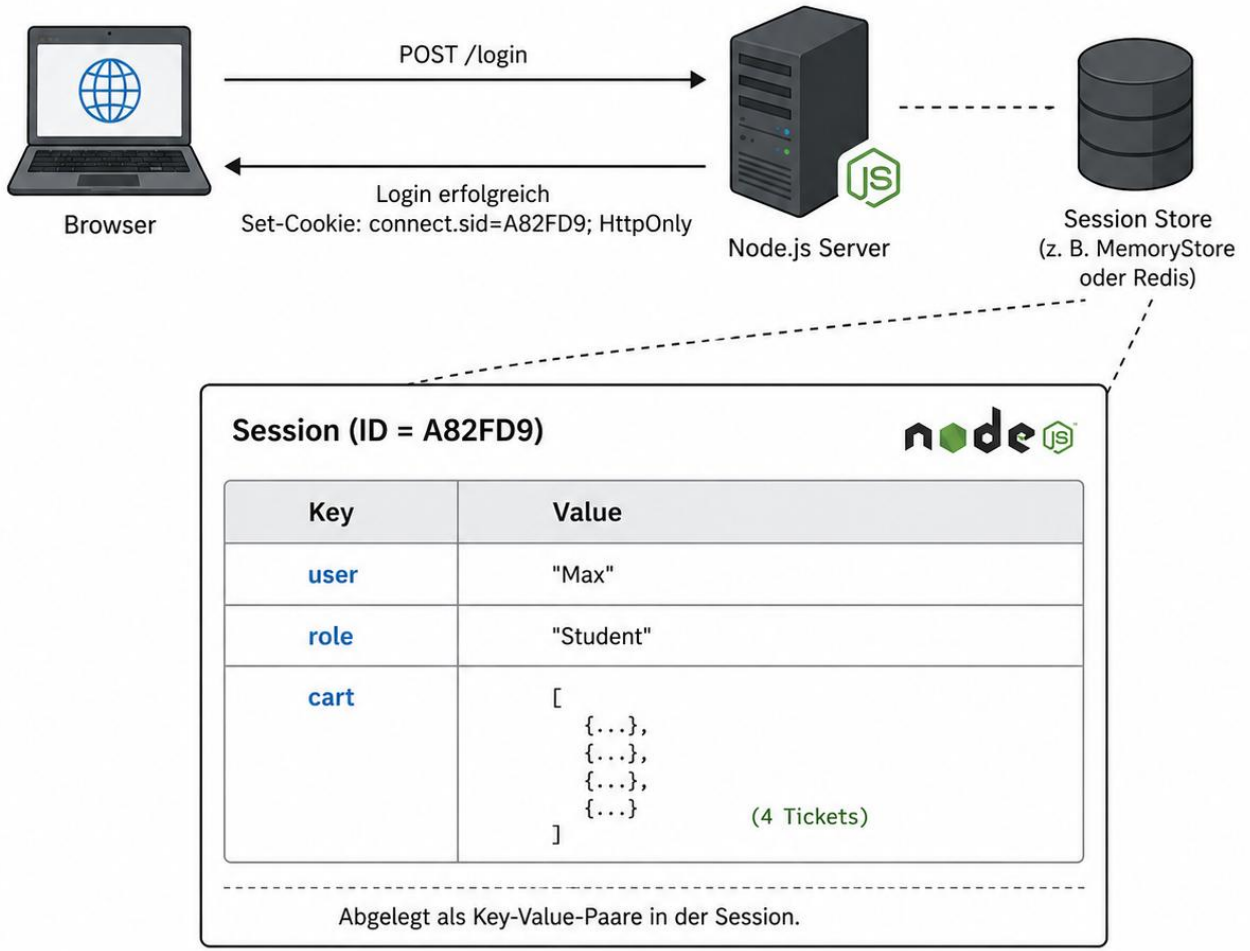
WIR BENÖTIGEN EINE EINDEUTIGE KENNUNG

Der Server vergibt deshalb für jede Anmeldung eine eindeutige Session-ID.

Die Session enthält alle Informationen, die während einer Benutzersitzung benötigt werden.

Definition

Eine **Session** beschreibt den serverseitig gespeicherten Zustand eines Benutzers über mehrere HTTP-Anfragen hinweg.



HAW HAMBURG

DAS PROBLEM IST NOCH NICHT GELÖST

Jetzt besitzt der Server zwar eine Session.

Aber …

Woher weiß der Browser,
dass seine Session

A82FD9

heißt?

Und wie gelangt diese ID bei jeder Anfrage wieder zurück?

COOKIES

Cookies transportieren die Session-ID

Nach erfolgreichem Login sendet der Server

```
HTTP/1.1 200 OK

Set-Cookie:

connect.sid=A82FD9
```

Der Browser speichert dieses Cookie automatisch.

Bei jeder weiteren Anfrage wird es wieder mitgesendet.

```
GET /checkout

Cookie:

connect.sid=A82FD9
```

Jetzt kann der Server die richtige Session finden.

SESSION UND COOKIE

Wer speichert eigentlich was?

Browser	Server
Cookie	Session
Session-ID	Benutzerdaten
kleine Zeichenkette	Rolle
keine Passwörter	Warenkorb
keine Benutzerdaten	Loginstatus

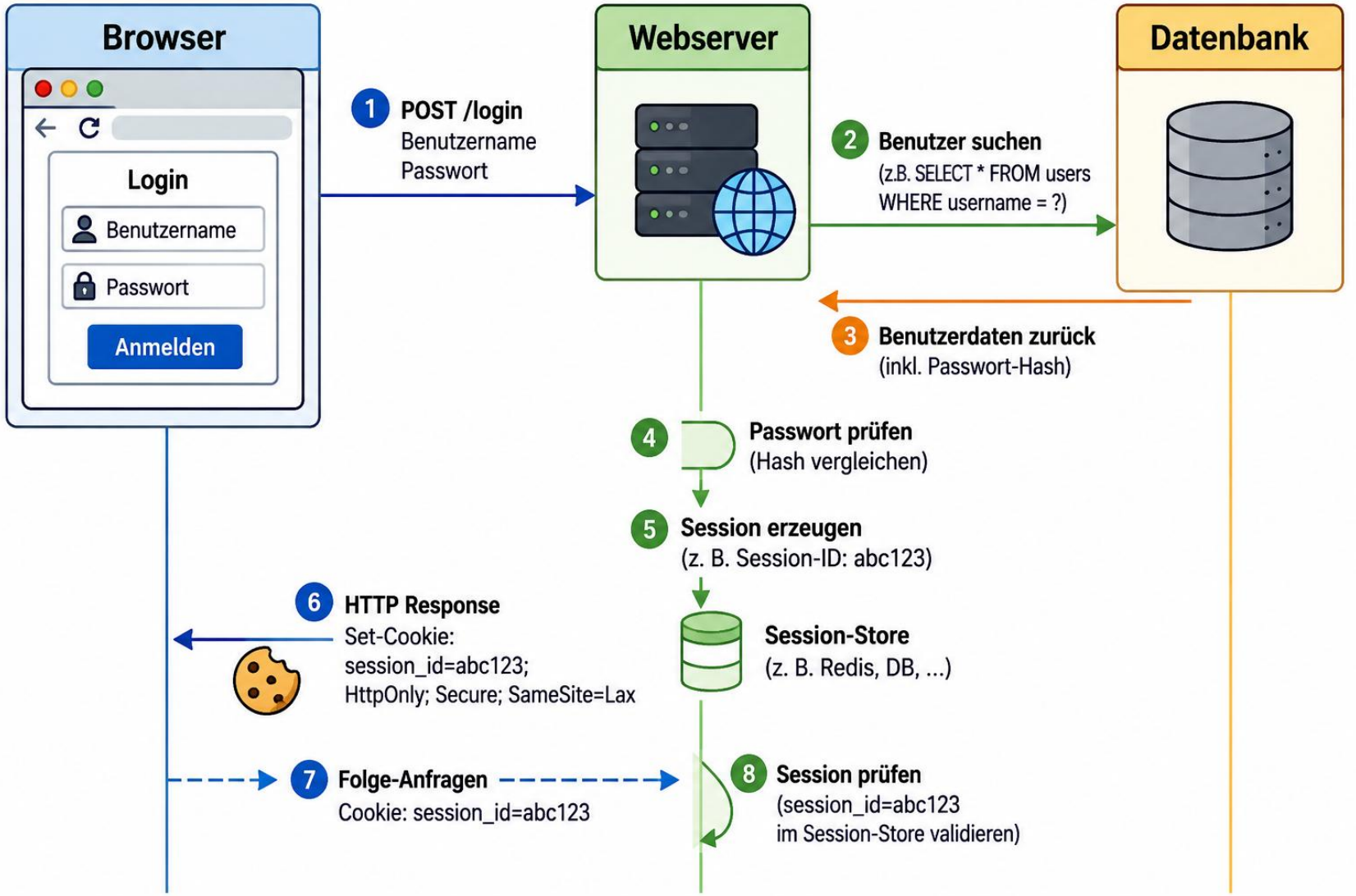
Merksatz

Das Cookie enthält **nicht** die Session.

Es enthält lediglich den **Schlüssel**, mit dem der Server die passende Session findet.

ZUSAMMENSPIEL

Jetzt erkennt der Server den Benutzer wieder.



WAS IST EINE SESSION?

SESSION – DER SERVERSEITIGE ZUSTAND EINES BENUTZERS

Definition

Eine **Session** ist ein serverseitig gespeicherter Datensatz, der Informationen über einen Benutzer während seiner Interaktion mit einer Webanwendung enthält. Die Session ermöglicht es, mehrere HTTP-Anfragen derselben Benutzerinteraktion zuzuordnen und den aktuellen Zustand der Anwendung zu erhalten.

Eine Session wird in der Regel nach erfolgreicher Authentifizierung angelegt und nach einer bestimmten Zeit der Inaktivität oder durch einen Logout wieder beendet.

Typische Informationen in einer Session

- Benutzer-ID (userId)
- Benutzername
- Rolle oder Berechtigungen
- Warenkorb
- Spracheinstellungen
- CSRF-Token
- Individuelle Benutzereinstellungen …

Wichtig: Passwörter werden **nicht** in einer Session gespeichert.

SESSIONS IN EXPRESS

Zugriff auf Sessions in Express

Nach der Konfiguration von express-session stellt Express für jede Anfrage das Objekt req.session bereit.

```
app.get("/profile", (req, res) => {

    console.log(req.session);

});
```

Eine Session kann beliebige Daten enthalten.

```
req.session.user = {
    id: 17,
    username: "max",
    role: "student"
};

req.session.language = "de";

req.session.cart = [
    12,
    18,
    25
];
```

Praxis

`req.session` verhält sich wie ein normales JavaScript-Objekt.

Änderungen werden automatisch im Session Store gespeichert.

WAS IST EIN COOKIE?

COOKIE – KLEINE INFORMATIONEN IM BROWSER

Definition

Ein **Cookie** ist ein kleiner Datensatz, den ein Webserver im Browser eines Benutzers speichern kann. Cookies werden automatisch bei zukünftigen HTTP-Anfragen an dieselbe Domain übertragen.

Cookies dienen unter anderem dazu,

- eine Session-ID zu speichern,
- Spracheinstellungen zu merken,
- Benutzereinstellungen zu speichern,
- Anmeldeinformationen über mehrere Anfragen hinweg bereitzustellen.

Eigenschaften eines Cookies

- Speicherung im Browser
- maximale Größe ca. **4 KB**
- Übertragung bei jeder passenden HTTP-Anfrage
- Lebensdauer durch Expires oder Max-Age

Merke: Ein Cookie speichert **nicht automatisch** Benutzerdaten. In vielen Anwendungen enthält es lediglich die Session-ID.

COOKIES IN EXPRESS

Cookies setzen und lesen

Ein Cookie kann direkt über die HTTP-Antwort gesetzt werden.

```
res.cookie("theme", "dark");
```

Oder mit zusätzlichen Optionen:

```
res.cookie("theme", "dark", {
    httpOnly: true,
    secure: true,
    sameSite: "lax",
    maxAge: 1000 * 60 * 60
});
```

Ein Browser sendet das Cookie anschließend automatisch bei jeder passenden Anfrage mit.

COOKIES UND SESSION IN EXPRESS

npm install express-session

```
import session from "express-session";

app.use(session({

    secret: "Ganz geheimes Passwort!",
    resave: false,
    saveUninitialized: false,

    cookie: {
        httpOnly: true,
        sameSite: "lax",
        secure: false
    }
}));
```

Erklärung

secret
Signiert die Session-ID.

resave = false
Speichert Sessions nur bei Änderungen erneut.

saveUninitialized = false
Leere Sessions werden nicht gespeichert.

Die Bibliothek erzeugt und verwaltet das Session-Cookie automatisch. Der Entwickler muss die Session-ID in der Regel nicht selbst setzen oder auslesen.

SESSIONS UND COOKIES IM VERGLEICH

Session	Cookie
liegt auf dem Server	liegt im Browser
enthält Benutzerdaten	enthält meist nur die Session-ID
beliebig groß (abhängig vom Session Store)	ca. 4 KB
kann nicht direkt vom Benutzer eingesehen werden	kann in den Browser-Entwicklertools angezeigt werden
endet durch Timeout oder Logout	endet durch Ablaufzeit oder Löschen

SICHERE COOKIES

Nicht jedes Cookie eignet sich für Sessions.

Attribut	Zweck
HttpOnly	verhindert Zugriff durch JavaScript
Secure	nur über HTTPS
SameSite	Schutz vor CSRF
Max-Age	Lebensdauer

Praxis

Session-Cookies sollten grundsätzlich

- HttpOnly
- Secure
- SameSite

verwenden.

ZUSAMMENFASSUNG

- HTTP ist zustandslos.
- Der Server erkennt Benutzer nach einer neuen Anfrage nicht automatisch wieder.
- Eine Session speichert den Anmeldezustand auf dem Server.
- Das Cookie enthält lediglich die Session-ID.
- Erst das Zusammenspiel von Session und Cookie ermöglicht einen dauerhaften Login.

AUTHENTIFIZIERUNG MIT PASSPORT.JS

AUTHENTIFIZIERUNG MIT PASSPORT.JS

Lernziele

Nach diesem Kapitel könnt ihr…

- erklären, warum Authentifizierungsframeworks eingesetzt werden,
- das Strategy Pattern am Beispiel von Passport erläutern,
- Passport in eine Express-Anwendung integrieren,
- eine Local Strategy implementieren,
- verstehen, wie Passport mit Sessions zusammenarbeitet.

MOTIVATION

Unser Login wächst

Im letzten Kapitel konnten sich Benutzer erfolgreich anmelden.

Unser Login besteht inzwischen aus:

- Passwortprüfung mit bcrypt
- Sessions
- Cookies
- Session Store
- Sicherheitskonfiguration

Je größer die Anwendung wird, desto mehr Authentifizierungslogik entsteht.

Fragen, die wir zusätzlich beantworten müssen:

- Wie erfolgt der Login?
- Wie erfolgt der Logout?
- Wie werden Sessions wiederhergestellt?
- Wie können später Google- oder GitHub-Login integriert werden?

Leitfrage: Müssen wir all diese Funktionen selbst entwickeln?

WARUM PASSPORT?

Authentifizierung als wiederverwendbare Komponente

Passport ist eine Middleware für Express, die typische Aufgaben der Authentifizierung übernimmt. Ziel ist es nicht, die Anwendung vollständig zu automatisieren, sondern wiederkehrende Authentifizierungsprozesse zu vereinheitlichen.

Passport unterstützt unter anderem:

- lokale Anmeldung mit Benutzername und Passwort,
- Anmeldung über externe Identitätsanbieter (z. B. Google oder GitHub),
- Wiederherstellung angemeldeter Benutzer aus einer Session.

Die eigentliche Benutzerverwaltung (Datenbank, Rollen, Berechtigungen) bleibt weiterhin Aufgabe der Anwendung.

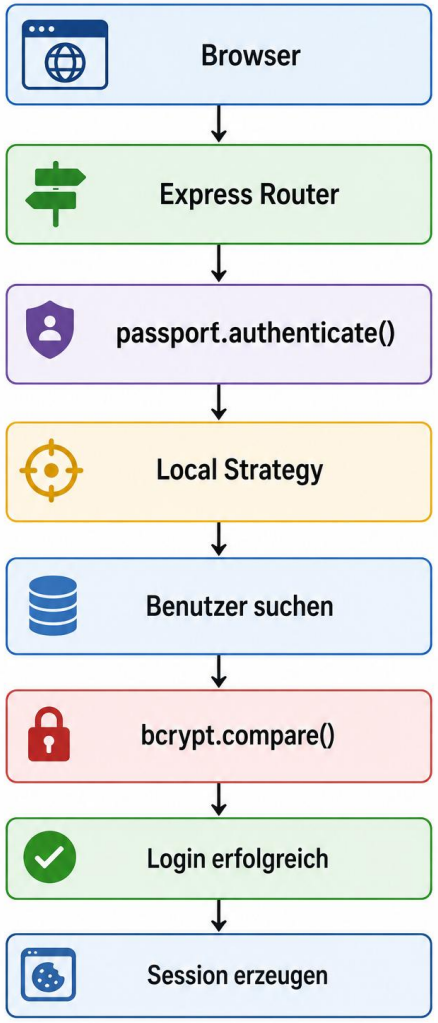
Merke: Passport ersetzt nicht den Login – es strukturiert ihn.

ARCHITEKTUR

Passport innerhalb unserer Anwendung

Passport sitzt **zwischen Route und Authentifizierungslogik**.

Dadurch wird die Authentifizierung unabhängig vom eigentlichen Anwendungscode.



WAS IST EINE STRATEGY?

Das Strategy Pattern

Passport basiert auf dem **Strategy Pattern**.
Eine Strategy beschreibt, **wie** ein Benutzer authentifiziert wird.

Beispiele:

Strategy	
Local Strategy	Benutzername und Passwort
Google Strategy	Anmeldung mit Google
GitHub Strategy	Anmeldung mit GitHub
Facebook Strategy	Anmeldung mit Facebook
JWT Strategy	APIs ohne Session

Alle Strategien besitzen dieselbe Schnittstelle. Dadurch kann die Authentifizierung erweitert werden, ohne bestehende Routen grundlegend zu verändern.

LOCAL STRATEGY

Authentifizierung mit Benutzername und Passwort

Installation

```
npm install passport passport-local
```

Passport übernimmt die Steuerung des Login-Prozesses. Die Anwendung entscheidet weiterhin selbst, wie Benutzer aus der Datenbank gelesen und Passwörter überprüft werden.

Registrierung einer Local Strategy

```
passport.use(new LocalStrategy(
    async (username, password, done) => {

        const user = await User.findOne({ username });

        if (!user)
            return done(null, false);

        const ok =
            await bcrypt.compare(
                password,
                user.password
            );

        if (!ok)
            return done(null, false);

        return done(null, user);

    }
));
```

SERIALIZEUSER() UND DESERIALIZEUSER()

Warum benötigt Passport diese Funktionen?

Nach erfolgreicher Authentifizierung muss entschieden werden, welche Informationen in der Session gespeichert werden.

```
passport.serializeUser((user, done) => {
    done(null, user.id);
});
```

Hier wird lediglich die Benutzer-ID in der Session gespeichert.

Bei einer späteren Anfrage wird daraus wieder ein vollständiges Benutzerobjekt geladen.

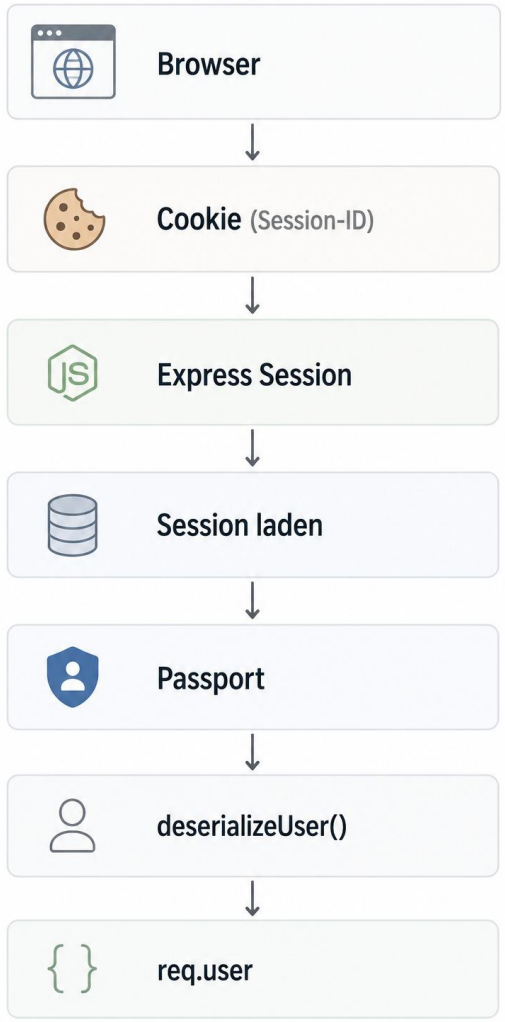
```
passport.deserializeUser(
    async (id, done) => {
        const user = await User.findById(id);
        done(null, user);
});
```

Dadurch bleibt die Session klein und aktuell.

WAS PASSIERT BEI EINER NEUEN ANFRAGE?

Passport liest den Benutzer automatisch aus der Session und stellt ihn in jeder Anfrage als req.user zur Verfügung.

Dadurch muss der Entwickler den angemeldeten Benutzer nicht bei jeder Route erneut aus der Datenbank laden.



AUTHENTIFIZIERUNG SCHÜTZEN

Passport stellt Middleware bereit.

```
function isAuthenticated(req, res, next) {

    if (req.isAuthenticated()) {
        return next();
    }

    res.redirect("/login");
}
```

Verwendung:

```
app.get(
    "/profile",
    isAuthenticated,
    controller.profile
);
```

Die Route wird nur ausgeführt, wenn ein Benutzer angemeldet ist.

VORTEILE UND GRENZEN VON PASSPORT

Vorteile	Grenzen
Klare Struktur	Zusätzliche Bibliothek
Viele Strategien verfügbar	Konfiguration anfangs ungewohnt
Gute Integration in Express	Kein Ersatz für Rollen- oder Rechteverwaltung
Einheitliche Authentifizierung	Benutzerverwaltung bleibt Aufgabe der Anwendung

Passport nimmt Entwicklern wiederkehrende Aufgaben ab, ersetzt aber nicht das Sicherheitskonzept der Anwendung.

ZUSAMMENFASSUNG

Das haben wir gelernt

- Passport ist eine Middleware für Express zur Strukturierung der Authentifizierung.
- Unterschiedliche **Strategien** ermöglichen verschiedene Login-Verfahren.
- Die **Local Strategy** nutzt weiterhin Benutzername, Passwort und bcrypt.
- serializeUser() legt fest, welche Informationen in der Session gespeichert werden.
- deserializeUser() stellt den angemeldeten Benutzer als req.user bereit.
- Passport baut auf den bereits bekannten Konzepten **Sessions** und **Cookies** auf und ersetzt sie nicht.